Studio Nx

Git · GitHub 관리 매뉴얼

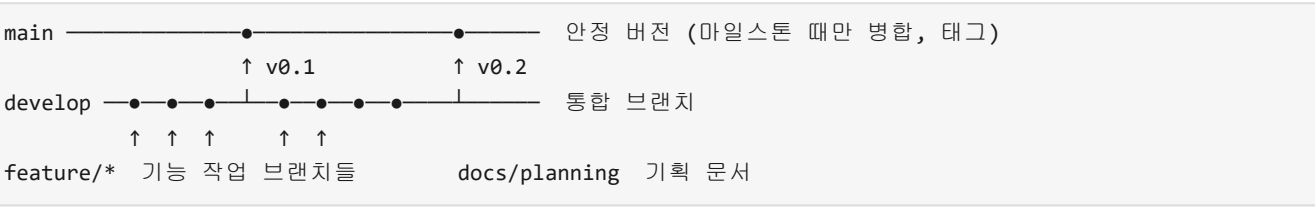
2026-10-01

Studio Nx — Git · GitHub 관리 매뉴얼

항목	내용
저장소	https://github.com/good200005/StudioNx (**비공개**)
기본 브랜치	`main`
작성일	2026-10-02

---

1. 브랜치 구조



브랜치	용도	병합 대상
`main`	안정 버전	—
`develop`	기능 통합	`main` (마일스톤 때)
`feature/phase1-account`	회원·로그인·마이페이지·프로젝트 CRUD	`develop`
`feature/phase2-sequencer`	시퀀서 코어	`develop`
`feature/phase3-export`	오디오 출력	`develop`
`feature/phase4-separation`	악기 분리	`develop`
`feature/phase5-ai`	AI 분석 · Nyxi 도우미 · 팁	`develop`
`feature/phase6-collab`	협업 · 톡방 · 코멘트 · 스냅샷	`develop`
`feature/native-dsp`	C++/JUCE → WASM	`develop`
`docs/planning`	PRD · DB · 구현 계획 등 문서	`develop` → `main`

- 큰 기능은 하위 브랜치로 나눕니다: `feature/phase2-sequencer/automation`
- 버그 수정: `fix/<내용>` (예: `fix/login-redirect`), `develop`에서 만들고 `develop`에 병합

2. 기본 작업 흐름

```
git switch develop
git pull                                  # 1) 최신 develop 받기
git switch feature/phase1-account
git merge develop                         # 2) 내 브랜치를 최신 상태로
# ... 작업 ...
git add -A
git commit -m "feat(auth): 회원가입 API 추가"   # 3) 커밋
git push                                  # 4) 원격에 올리기
```

5. GitHub에서 **Pull Request** 생성: `feature/...` → `develop`
6. 변경 내용 확인 후 **Merge** (혼자 개발해도 PR로 기록을 남깁니다)

마일스톤 (예: v0.1 MVP)

```
git switch main
git pull
git merge --no-ff develop
git tag -a v0.1 -m "v0.1 MVP"
git push --follow-tags
```

문서만 고칠 때

`docs/planning`에서 커밋 → PR로 `develop`에 병합 → 문서는 바로 `main`에도 반영해도 됩니다.

3. 커밋 메시지 규칙

형식: `타입(범위): 내용` — 내용은 한국어, 무엇을 했는지 한 줄로.

타입	언제	예시
`feat`	기능 추가	`feat(sequencer): 트랙 아코디언 펼치기`
`fix`	버그 수정	`fix(auth): 토큰 만료 시 재발급 안 되던 문제`
`docs`	문서	`docs(prd): Phase 2 오토메이션 요구사항 추가`
`db`	스키마·SQL	`db: user_settings 에 tips_level 추가`
`style`	HTML/CSS 디자인	`style(home): 히어로 섹션 레이아웃`
`refactor`	동작 변화 없는 정리	`refactor(api): 라우터 파일 분리`
`test`	테스트	`test(audio): 패처 회귀 테스트 추가`
`chore`	설정·의존성	`chore: sentry-sdk 버전 업데이트`

- 한 커밋에는 한 가지 목적만 담습니다.
- AI가 작성한 커밋에는 `Co-Authored-By: Claude ...` 줄이 붙습니다.

4. 자주 쓰는 명령

목적	명령
현재 상태	`git status`
브랜치 목록	`git branch -a`
브랜치 이동	`git switch <브랜치>`
새 브랜치 만들고 이동	`git switch -c fix/<내용> develop`
최신 받기	`git pull`
변경 내용 보기	`git diff`
기록 보기	`git log --oneline --graph -15`
작업 임시 보관 / 꺼내기	`git stash` / `git stash pop`
원격 브랜치 정리	`git fetch --prune`

5. 실수했을 때

상황	해결
마지막 커밋 메시지 수정 (아직 push 전)	`git commit --amend -m "새 메시지"`
커밋을 되돌리기 (push 후)	`git revert <커밋ID>` — 기록을 지우지 않고 반대 커밋 생성
파일 수정 취소 (커밋 전)	`git restore <파일>`
스테이징 취소	`git restore --staged <파일>`
병합 충돌	충돌 파일에서 `<<<<<<<` ~ `>>>>>>>` 부분 정리 → `git add <파일>` → `git commit`
비밀번호·키를 커밋해 버림	**즉시 그 비밀번호를 변경**하고 파일을 `.gitignore`에 추가. 기록에서 지우는 것만으로는 안전하지 않음

6. 하지 말 것

- `main`에 직접 커밋·push 하지 않기 (마일스톤 병합만)
- `git push --force` 를 공유 브랜치(`main`, `develop`)에 쓰지 않기
- `.env`, `.venv/`, `storage/`, `node_modules/` 커밋하지 않기 (이미 `.gitignore`에 있음)
- 큰 오디오 파일(샘플·출력물)을 저장소에 넣지 않기 — 테스트용 골든 파일만 `studionx/tests/audio/golden/`에

7. GitHub 관리

작업	방법
다른 사람에게 공개	저장소 → **Settings → Collaborators → Add people** (GitHub 계정 필요). 비공개 저장소라 링크만으로는 볼 수 없음
공개 범위 변경	**Settings → General → Danger Zone → Change visibility**. 공개 전에 `db/00_create_database.sql`의 비밀번호 변경 필수
`main` 보호	**Settings → Branches → Add rule**: `main`에 "Require a pull request before merging"
기본 브랜치 변경	**Settings → General → Default branch** (예: `develop`)
이슈로 할 일 관리	**Issues**에 작업 등록 (예: `S1-02 인증 API`), PR 설명에 `Closes #번호` 적으면 병합 시 자동 종료